

University of Windsor

COMP - 8977 Internship/Project-2

Fall 2026

Project Proposal

Explainable Industrial Anomaly Detection using Vision-Language Models

Team 12

Supervisor: Dr. Shaon Shuvo

Course Instructor: Dr. Prashanth Cheluvasai Ranga

Submitted By

Prabhjot Singh	110195228
Sarhan Kapadiya	110209974

Table of Contents

1. Introduction	3
2. Motivation	3
3. Background Study	4
4. Project Goals	5
5. Project Roadmap	7
6. Resources	9
7. References	10

1. Introduction

Automated visual inspection is central to modern manufacturing. Cameras on production lines capture thousands of images of fabrics, tiles, metal parts, and electronics, and machine-learning models are expected to flag the ones that contain defects such as scratches, cracks, holes, or contamination. Industrial anomaly detection is difficult because defects are rare, varied, and often subtle, so most methods are trained on normal images only and treat anything that deviates from normal as an anomaly.

Vision-Language Models (VLMs) such as CLIP have recently changed this field. Because they are trained on large collections of image-text pairs, they can detect anomalies in zero-shot or few-shot settings, that is, with no defective training images at all, by comparing an image against text descriptions such as a normal object versus a damaged object. Methods like WinCLIP and AnomalyCLIP follow this idea and reach strong results on industrial benchmarks without per-product training.

These models still behave as black boxes. A detector that returns "anomalous, score 0.93" gives an engineer little to act on and little reason to trust it, especially in safety-critical production. Our project therefore focuses on explainability. We will build a VLM-based anomaly detection pipeline and then explain each decision in two complementary ways: a visual explanation (an attribution map showing which image regions drove the decision) and a language explanation (a short natural-language description of the suspected defect, its location, and the evidence for it). The central contribution is a quantitative comparison of explanation methods, including a baseline-search idea adapted from ABIGX (Zhuo et al., 2026), against known defect masks.

2. Motivation

2.1 Why is this Problem Interesting?

The problem sits where three active research areas meet: foundation vision-language models, industrial inspection, and explainable AI (XAI). Zero-shot VLM detectors remove the largest practical obstacle in inspection, the lack of labelled defect data, but their reasoning is even less transparent than that of a small task-specific network. They rely on a large pre-trained backbone and on text prompts whose effect on the final score is hard to predict.

Explainability matters here for a concrete reason. A recent review of XAI for industrial fault detection (Maged et al., 2026) concludes that trust in black-box models is a main barrier to adoption in safety-critical settings. The same review points out that studies rarely evaluate explanations with standardized quantitative metrics, and that few examine how operators actually use explanations. Our project addresses both gaps on a concrete visual inspection task.

2.2 Why is it Challenging?

1. **Differentiability and access:** Attribution methods need gradients through the detector. VLM anomaly detectors often use non-differentiable steps such as thresholding or cropping, so the pipeline must be adapted carefully without changing its behaviour.
2. **Baseline choice:** Integrated Gradients depends on a reference image. A black or constant image is semantically meaningless for a texture or a product photo and tends to give scattered attributions. Finding a meaningful normal-looking baseline for each defective image is an open design question.

3. **Small, fine-grained defects:** Defects may cover a tiny part of the image. Patch-level backbones such as ViT-B/16 have coarse spatial resolution, so heatmaps can look blurry or miss thin defects.
4. **Evaluating explanations:** A heatmap that looks plausible is not necessarily faithful to the model. We need metrics that test both agreement with ground truth (correctness) and agreement with model behaviour (consistency).
5. **Trustworthy language explanations:** Generative VLMs can describe defects that are not there. Text explanations must be grounded in the detector's evidence and checked for hallucination.

2.3 Who Benefits?

- **Quality engineers and line operators:** Visual and textual explanations show where and why an item was flagged, so false alarms can be dismissed quickly and real defects acted on.
- **Manufacturers:** Zero-shot detection with explanations lowers the cost of deploying inspection on new product lines that have no defect data.
- **Researchers:** A reproducible comparison of explanation methods on VLM detectors, with quantitative metrics, supports further work on trustworthy industrial AI.
- **Students and educators:** The project is a hands-on case study connecting foundation models, computer vision, and XAI.

3. Background Study

3.1 Industrial Anomaly Detection and MVTec AD

The MVTec Anomaly Detection (MVTec AD) dataset (Bergmann et al., 2019) is the standard benchmark for unsupervised industrial anomaly detection. It contains high-resolution images of 15 object and texture categories, including carpet, leather, tile, wood, grid, bottle, and transistor. The training set has only defect-free images. The test set mixes normal images with defective ones covering many defect types, and the defective images come with pixel-accurate ground-truth masks. These masks are what allow us to test whether an explanation points at the real defect. Memory-bank methods such as PatchCore (Roth et al., 2022) are strong non-VLM references and will serve as a comparison point for detection quality.

3.2 Vision-Language Models for Anomaly Detection

CLIP (Radford et al., 2021) learns a shared embedding space for images and text from large-scale image-text pairs. WinCLIP (Jeong et al., 2023) adapts CLIP to anomaly detection. It builds a compositional ensemble of text prompts describing normal and anomalous states, scores image windows at several scales to localize defects, and can add a few normal reference images for few-shot detection. AnomalyCLIP (Zhou et al., 2024) learns object-agnostic text prompts so that one model can generalize across categories. AnomalyGPT (Gu et al., 2024) goes further and uses a large vision-language model that answers questions about the image and outputs localization, which is close to the language explanations we want. We will use WinCLIP as the primary detector, since ABIGX has already been evaluated on it, and AnomalyCLIP as a second detector.

3.3 Explainability for Visual Models

Gradient-based attribution assigns an importance score to every input pixel or patch. The simplest method, a saliency map, is the gradient of the model output with respect to the input. Grad-CAM (Selvaraju et al., 2017) combines gradients with feature maps to produce coarse class-discriminative heatmaps. Integrated Gradients (IG) (Sundararajan et al., 2017) accumulates gradients along a path from a baseline image to the input and satisfies a completeness property, but its quality depends on the baseline. For transformers, relevance can also be derived from attention (Chefer et al., 2021).

ABIGX (Zhuo et al., 2026) addresses the baseline problem for fault detection and classification. Its first stage, Adversarial Fault Reconstruction (AFR), uses projected gradient descent to find a minimally changed version of the defective input that the model considers normal. Its second stage runs IG from this baseline to the original input. On the WinCLIP detector and MVTec AD, the authors report more focused attribution maps and higher correctness scores than saliency maps, Expected IG, and vanilla IG, at roughly twice the gradient cost of standard IG. They also report limitations: AFR can get stuck in poor local optima, the baseline is only an approximate projection onto the normal region, and the attributions are associational rather than causal. We will re-implement the idea for our detector instead of copying the authors' code, and we will test it against the baselines.

3.4 Evaluating Explanations

We follow the two metric families used in ABIGX. Correctness measures agreement between the attribution map and the ground-truth defect mask, either as the ROC-AUC of attribution scores treated as pixel classifiers or as the share of attribution mass inside the mask. Consistency measures agreement with model behaviour: in the ADD test, the most important regions of the defective image are progressively pasted into a normal image and the anomaly score should rise quickly, while in the DEL test the most important regions are progressively removed from the defective image and the score should fall quickly. For language explanations we add defect-type accuracy against the MVTec defect labels, region-grounding agreement, a hallucination rate, and a small human rating study.

4. Project Goals

4.1 Overall Vision

We aim to build and evaluate an end-to-end explainable anomaly detection pipeline for industrial images. A VLM detector produces an anomaly score and map, explanation modules produce visual and language explanations, and an evaluation suite measures how faithful and accurate they are. The focus is a well-evaluated, reproducible comparison, not leaderboard-level detection performance. Figure 1 shows the planned pipeline.

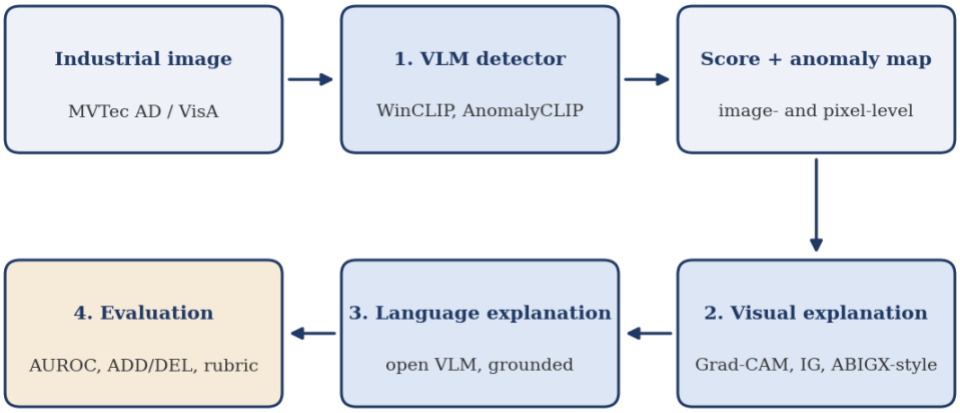


Figure 1. Planned explainable anomaly detection pipeline.

4.2 Specific Objectives

1. **Build the detection pipeline:** Run WinCLIP (zero-shot and few-shot) and AnomalyCLIP on MVTec AD and report image-level AUROC, pixel-level AUROC, and PRO per category, with PatchCore as a non-VLM reference.
2. **Implement visual explanations:** Provide saliency, Grad-CAM or attention-based relevance, Integrated Gradients with black and normal-sample baselines, and an ABIGX-style explainer (AFR baseline search followed by IG) on the same detector.
3. **Generate grounded language explanations:** Use an open-source VLM, given the image, the detector's score, and the highlighted region, to describe the suspected defect type, location, and supporting evidence.
4. **Evaluate explanations quantitatively:** Report Correctness (AUC and SUM) and Consistency (ADD and DEL) for every visual method, and defect-type accuracy, region grounding, and hallucination rate for language explanations, plus runtime per image.
5. **Deliver a usable prototype and documentation:** Provide a small web demo (upload an image, see score, heatmap, and explanation) together with reproducible code and a final IEEE-style report.

4.3 What We Are NOT Committing To

To keep the project feasible within one term, we explicitly exclude the following:

- Training or fine-tuning a vision-language model from scratch; we use pre-trained checkpoints and only lightweight prompt learning where a method requires it
- Matching state-of-the-art detection scores on MVTec AD; the goal is explanation quality, not leaderboard rank
- Collecting a new industrial dataset or running experiments on a physical production line
- Real-time or production-scale deployment; the demo is prototype scale

- Causal root-cause analysis; our explanations indicate where the model looks, not why the defect physically occurred

4.4 Risks and Mitigation

Risk	Impact	Mitigation
Detector is hard to make differentiable	Gradient-based explanations cannot run	Start from a PyTorch WinCLIP implementation and replace non-differentiable steps with smooth equivalents; verify scores are unchanged
AFR baseline search fails on some images	Noisy ABIGX-style attributions	Monitor final anomaly score, then fall back to Expected IG with normal images as baselines, as the ABIGX authors suggest
Limited GPU memory or time	Slow experiments, fewer categories	Run a subset of categories first, use batching and half precision, and report timing
VLM text explanations hallucinate	Misleading language output	Ground prompts in the attribution region and detector score, constrain the output format, and measure hallucination rate

5. Project Roadmap

The project runs for eight weeks of the Fall 2026 term, from October 12 to December 4, 2026. Each team member has dedicated tasks every week with a distinct, assessable deliverable. Both members contribute equally to the experiments and the final report.

Week 1 (Oct 12–18): Foundation & Proposal

Team Member	Task	Deliverable
Prabhjot Singh	Write the project proposal (introduction, motivation, goals); review VLM anomaly detection papers (WinCLIP, AnomalyCLIP)	Proposal draft; literature summary (detectors)
Sarhan Kapadiya	Set up GitHub repository, environment (PyTorch, open_clip, Captum), and branching conventions; review XAI methods (Grad-CAM, IG, SHAP)	Repository and environment; literature summary (XAI)

Week 2 (Oct 19–25): Data & Detector Setup

Team Member	Task	Deliverable
Prabhjot Singh	Download and verify MVTec AD; build data loaders and the metric scripts (image AUROC, pixel AUROC, PRO)	Dataset loader and detection metrics

Team Member	Task	Deliverable
Sarhan Kapadiya	Reproduce WinCLIP zero-shot inference with prompt ensembles on the selected categories	Working WinCLIP zero-shot pipeline

Week 3 (Oct 26–Nov 1): Detection Baselines

Team Member	Task	Deliverable
Prabhjot Singh	Add few-shot WinCLIP (1, 2, and 4 normal reference images) and run a prompt ablation	WinCLIP few-shot results table
Sarhan Kapadiya	Run AnomalyCLIP inference and a PatchCore reference baseline	AnomalyCLIP and PatchCore results

Week 4 (Nov 2–8): Visual Explanation Baselines

Team Member	Task	Deliverable
Prabhjot Singh	Make the detector differentiable end to end; implement saliency and Integrated Gradients (black and Expected-IG baselines) with Captum	Attribution module v1
Sarhan Kapadiya	Implement Grad-CAM or attention-based relevance for the ViT backbone; build heatmap overlay and visualization tools	Relevance module and visualization tool

Week 5 (Nov 9–15): ABIGX-Style Explainer & Metrics

Team Member	Task	Deliverable
Prabhjot Singh	Implement AFR baseline search (PGD on the anomaly score) followed by IG along the path; add convergence checks and fallback	ABIGX-style explainer
Sarhan Kapadiya	Implement explanation metrics: Correctness (AUC, SUM against masks) and Consistency (ADD, DEL)	Explanation metrics suite

Week 6 (Nov 16–22): Comparison & Language Explanations

Team Member	Task	Deliverable
Prabhjot Singh	Run all explainers across categories; ablate AFR steps, step size, and baseline type	Explanation benchmark report
Sarhan Kapadiya	Build the VLM prompting module: image, score, and highlighted region in; structured explanation (defect type, location, evidence) out	Language explanation module

Week 7 (Nov 23–29): Language Evaluation & Demo

Team Member	Task	Deliverable
Prabhjot Singh	Evaluate language explanations: defect-type accuracy, region grounding, hallucination rate; analyse failure cases; rate a sample subset	Language explanation evaluation report
Sarhan Kapadiya	Build the demo interface (upload image, view score, heatmap, and explanation); rate a sample subset with the shared rubric	Working demo; human rating results

Week 8 (Nov 30–Dec 4): Final Report & Presentation

Team Member	Task	Deliverable
Prabhjot Singh	Write IEEE report sections: Methodology, Experimental Setup, and Results	IEEE Methodology and Results
Sarhan Kapadiya	Write IEEE report sections: Introduction, Related Work, and Discussion; prepare final slides and demo walkthrough	IEEE Introduction and Discussion; final presentation deck

5.1 Individual Role Summary

The table below summarizes the primary area of ownership for each team member across the project.

Team Member	Primary Role	Key Contributions
Prabhjot Singh	Detection & Explainer Lead	Proposal, MVTec AD loaders and detection metrics, few-shot WinCLIP, differentiable detector, IG baselines, ABIGX-style explainer, explanation benchmark, language evaluation, Methodology and Results
Sarhan Kapadiya	Infrastructure & Evaluation Lead	Repository and environment, WinCLIP zero-shot pipeline, AnomalyCLIP and PatchCore, Grad-CAM, explanation metrics, VLM explanation module, demo, Introduction, Discussion, final slides

6. Resources

6.1 Required Skills

- **Python and deep learning:** PyTorch, NumPy, Pandas, and working with pre-trained transformer models
- **Computer vision:** Image preprocessing, patch-based features, segmentation metrics such as AUROC and PRO
- **Vision-language models:** CLIP-style image-text embeddings, prompt design, and open-source VLM inference
- **Explainable AI:** Gradient-based attribution, baselines, and faithfulness metrics

- **Version control and technical writing:** Git collaboration, experiment documentation, and IEEE-style reporting

6.2 Technology Stack

Component	Technology / Tool
Programming Language	Python 3.10+
Core Libraries	PyTorch, NumPy, Pandas, scikit-learn, OpenCV
VLM Detectors	WinCLIP (CLIP ViT-B/16+ backbone via open_clip), AnomalyCLIP; PatchCore as non-VLM reference
Explainability	Captum (saliency, Integrated Gradients), Grad-CAM implementation, custom AFR and ABIGX-style module
Language Explanations	Open-source vision-language model (for example LLaVA or a similar model) run locally or through an API
Datasets	MVTec AD (primary); VisA (optional, if time permits)
Evaluation Metrics	Image and pixel AUROC, PRO, Correctness (AUC, SUM), Consistency (ADD, DEL), defect-type accuracy, hallucination rate
Demo Interface	Streamlit or Gradio
Development Tools	Jupyter Notebook, Git / GitHub, pytest, Black / Ruff
Visualization	Matplotlib, Seaborn

6.3 Hardware Requirements

- **GPU:** CUDA-capable GPU with at least 8 GB of VRAM for ViT inference and gradient-based explanations (personal machine, university lab, or cloud notebook)
- **CPU and RAM:** Multi-core processor with at least 16 GB of RAM
- **Storage:** About 30 GB for datasets, model checkpoints, and saved attribution maps

6.4 Project Management Tools

- **GitHub Projects and Issues:** Task tracking, pull request reviews, and bug reports
- **Shared document repository:** Collaborative writing and file sharing for the report and slides
- **Communication:** Weekly team meetings and regular check-ins with the supervisor

7. References

1. Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2019). MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 9592–9600.
2. Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision. Proceedings of the International Conference on Machine Learning (ICML), 8748–8763.

3. Jeong, J., Zou, Y., Kim, T., Zhang, D., Ravichandran, A., & Dabeer, O. (2023). WinCLIP: Zero-/few-shot anomaly classification and segmentation. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 19606–19616.
4. Zhou, Q., Pang, G., Tian, Y., He, S., & Chen, J. (2024). AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection. International Conference on Learning Representations (ICLR).
5. Gu, Z., Zhu, B., Zhu, G., Chen, Y., Tang, M., & Wang, J. (2024). AnomalyGPT: Detecting industrial anomalies using large vision-language models. Proceedings of the AAAI Conference on Artificial Intelligence.
6. Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). Towards total recall in industrial anomaly detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
7. Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. Proceedings of the International Conference on Machine Learning (ICML), 3319–3328.
8. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. Proceedings of the IEEE International Conference on Computer Vision (ICCV).
9. Chefer, H., Gur, S., & Wolf, L. (2021). Transformer interpretability beyond attention visualization. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
10. Zhuo, Y., Qian, J., Zheng, J., Jiang, X., Song, Z., Chen, D., Yu, W., & Ge, Z. (2026). ABIGX: A unified framework for explainable fault detection and classification. IEEE Transactions on Pattern Analysis and Machine Intelligence, 48(9), 10906–10923.
11. Maged, A., Haridy, S., Hosny, M., & Shen, H. (2026). Explainable artificial intelligence in fault detection and diagnosis: a review of methods, applications, and implementation challenges. Journal of Industrial and Production Engineering, 43(5), 664–687.
12. Kokhlikyan, N., Miglani, V., Martin, M., et al. (2020). Captum: A unified and generic model interpretability library for PyTorch. arXiv preprint arXiv:2009.07896.
13. Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. Advances in Neural Information Processing Systems (NeurIPS), 36.